

React Native 学習まとめ

家計簿アプリ開発を通じて学んだ React Native の技術をまとめた資料です。

1. コンポーネント・Props・State

コンポーネントとは

画面のパーツを部品として切り出したものです。ボタン・テキスト・入力欄など、画面を構成するすべてのものがコンポーネントです。

```
export default function MyComponent() {  
  return <Text> こんにちは </Text>  
}
```

Props とは

親コンポーネントから子コンポーネントに値を渡す仕組みです。関数の引数のようなイメージです。

```
// 親  
<MyComponent name="太郎" />  
  
// 子  
type Props = {  
  name: string;  
}  
export default function MyComponent({ name }: Props) {  
  return <Text> {name} </Text>  
}
```

State とは

コンポーネントが持つ「変化する値」です。State が変わると画面が自動で再描画されます。

```
const [count, setCount] = useState(0);
```

```
// 更新するときは必ず setter 関数を使う  
setCount(count + 1);
```

2. よく使うコンポーネント

TextInput (テキスト入力)

```
const [text, setText] = useState('');
```

```
<TextInput placeholder="入力してください" value={text} onChangeText={(t) => setText(t)} secureTextEntry=
```

FlatList (リスト表示)

配列のデータをリスト形式で表示するコンポーネントです。ScrollView より効率的です。

```
<FlatList data={items} keyExtractor={({item}) => item.id} renderItem={({ item }) => <Text> {item.name}}
```

Button ・ Pressable

```
// シンプルなボタン
```

```
<Button title="送信" onPress={() => console.log('押した')} />
```

```
// カスタマイズしたいときは Pressable
```

```
<Pressable onPress={() => console.log('押した')}>
```

```
  <Text> アカウントをお持ちでない方はこちら </Text>
```

```
</Pressable>
```

注意： `onPress={fn()}` と書くと画面表示時に即実行されてしまう。
正しくは `onPress={() => fn()}` と書く。

3. StyleSheet ・ Flexbox

StyleSheet

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'white',
    padding: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
  },
})

<View style={styles.container}>
  <Text style={styles.title}> タイトル </Text >
</View >
```

Flexbox（レイアウト）

React Native のレイアウトは Flexbox が基本です。デフォルトは縦方向（column）です。

```
// 横並び
<View style={{ flexDirection: 'row' }}>

// 中央揃え
<View style={{ justifyContent: 'center', alignItems: 'center' }}>

// 残りのスペースを埋める
<View style={{ flex: 1 }}>
```

4. 画面遷移（Expo Router）

Expo Router はファイルのパスがそのままURLになります。

```
app/
├─ index.tsx      →  /
├─ login.tsx      →  /login
├─ signup.tsx     →  /signup
└─ (tabs)/
    └─ kakeibo.tsx →  /(tabs)/kakeibo
```

router の使い方

```
import { router } from 'expo-router';
```

```
// 画面を積み重ねる（戻るボタンで前の画面に戻れる）
```

```
router.push('/signup');
```

```
// 今の画面を置き換える（戻るボタンで戻れない）
```

```
router.replace('/login');
```

使い分けの目安：

- ログイン成功後 → `replace` （ログイン画面に戻らせたくない）
- ページ間の移動 → `push` （戻れた方が自然）

Stack.Screen（ヘッダーの設定）

```
<Stack >
  <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
  <Stack.Screen name="login" options={{ headerShown: false }} />
</Stack >
```

5. AsyncStorage（ローカル保存）

デバイスにデータを保存する仕組みです。アプリを閉じててもデータが残ります。

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// 保存（文字列のみ）
await AsyncStorage.setItem('key', JSON.stringify(data));

// 取得
const raw = await AsyncStorage.getItem('key');
const data = JSON.parse(raw ?? '[]');

// 削除
await AsyncStorage.removeItem('key');
```

6. CRUD 操作

CRUD とは Create（作成）・Read（読取）・Update（更新）・Delete（削除）の略です。

ローカル（State）での CRUD

```
// Create
setItems([...items, newItem]);

// Delete
setItems(items.filter((item) => item.id !== targetId));

// Update
setItems(items.map((item) =>
  item.id === targetId ? { ...item, name: '新しい名前' } : item
));
```

7. カテゴリ別集計

reduce（配列の合計を計算）

```
const total = items.reduce((acc, item) => acc + item.amount, 0);
```

Object.entries (オブジェクトをループ)

```
const categoryTotals = items.reduce((acc, item) => {
  acc[item.category] = (acc[item.category] ?? 0) + item.amount;
  return acc;
}, {} as Record<string, number>);

Object.entries(categoryTotals).forEach(([category, total]) => {
  console.log(`${category}: ${total}円`);
});
```

8. バリデーション

入力値が正しいかを確認する処理です。

```
if (inputText === '' || inputNumber === '') return;

if (Number(inputNumber) <= 0) {
  setErrorMessage('金額は1円以上を入力してください');
  return;
}
```

9. 日付管理

Date オブジェクト

```
const d = new Date();
d.getFullYear() // 2026
d.getMonth() + 1 // 月は0始まりなので +1 が必要
d.getDate() // 日
```

padStart (ゼロ埋め)

padStart は文字列メソッドです。String() で変換してから使います。

```
String(d.getMonth() + 1).padStart(2, '0') // "05"
```

月の絞り込み

```
const filteredItems = items.filter((item) =>
  item.date?.startsWith(`${year}-${String(month).padStart(2, '0')}`)
);
```

オプションチェーン (?.)

値が null や undefined のときにエラーにならないようにする書き方です。

```
item.date?.startsWith('2026') // date が null でもエラーにならない
```

10. グラフ表示 (react-native-chart-kit)

PieChart

```
import { PieChart } from 'react-native-chart-kit';
import { Dimensions } from 'react-native';

const screenWidth = Dimensions.get('window').width;

const data = [
  { name: '食費', population: 3000, color: '#f00', legendFontColor: '#333' },
  { name: '交際費', population: 1500, color: '#0f0', legendFontColor: '#333' },
];

<PieChart data={data} width={screenWidth} height={200} chartConfig={{ color: (opacity = 1) => `rgba(0,
```

% (モジュロ演算子)

カラーを順番に割り当てるときに使います。

```
const COLORS = ['#f00', '#0f0', '#00f'];
const color = COLORS[index % COLORS.length];
```

11. Supabase (バックエンド・データベース)

セットアップ

```
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!
);
```

CRUD

```
// Create
await supabase.from('expenses').insert({ name: '食費', amount: 500 });

// Read
const { data } = await supabase.from('expenses').select('*');

// Delete
await supabase.from('expenses').delete().eq('id', targetId);
```

エラーハンドリング

Supabase の関数はすべて { data, error } を返します。

```
const { data, error } = await supabase.from('expenses').insert({ ... });
if (error) {
  Alert.alert('エラー', error.message);
} else {
  // 成功時の処理
}
```


12. 認証 (Supabase Auth)

サインアップ

```
const { data, error } = await supabase.auth.signUp({
  email: email,
  password: password,
});
if (error) {
  Alert.alert('エラー', error.message);
} else {
  router.replace('/(tabs)/kakeibo');
}
```

ログイン

```
const { data, error } = await supabase.auth.signInWithPassword({
  email: email,
  password: password,
});
if (error) {
  Alert.alert('エラー', error.message);
} else {
  router.replace('/(tabs)/kakeibo');
}
```

ログアウト

```
await supabase.auth.signOut();
router.replace('/login');
```

13. ルート保護

ログインしていないユーザーをログイン画面に飛ばす仕組みです。 `_layout.tsx` に書きます。

```
useEffect(() => {
  const checkSession = async () => {
    const { data } = await supabase.auth.getSession();
    if (!data.session) {
      router.replace('/login');
    }
  }
  checkSession();
}, []);
```

14. useEffect

コンポーネントが表示されたタイミングで1回だけ実行したい処理に使います。

```
useEffect(() => {
  // 画面表示時に実行される
  loadItems();
}, []); // [] = 依存配列。空なら最初の1回だけ実行
```

注意： `useEffect` の中に `useState` などの Hook は書けない。
`onPress` の中に `useEffect` を書くのも誤り。

15. async / await

非同期処理（時間がかかる処理）を順番に実行するための書き方です。

```
// await をつけると処理が終わるまで待ってから次に進む
const { data } = await supabase.from('expenses').select('*');
```

注意： コンポーネント自体に `async` はつけられない。
`async` は関数に対してつける。

```
// 誤り
export default async function MyScreen() { ... }

// 正しい
export default function MyScreen() {
  const handlePress = async () => {
    await someAsyncFunction();
  }
}
```

よく使うパターン集

データ取得 → State に保存

```
const [items, setItems] = useState([]);

const loadItems = async () => {
  const { data } = await supabase.from('expenses').select('*');
  if (data) setItems(data);
}

useEffect(() => {
  loadItems();
}, []);
```

Alert (確認ダイアログ)

```
Alert.alert(
  '削除確認',
  '本当に削除しますか?',
  [
    { text: 'キャンセル', style: 'cancel' },
    { text: '削除', style: 'destructive', onPress: () => deleteItem() },
  ]
);
```

つまずきやすいポイント

ポイント	注意点
<code>onPress={fn()}</code>	即時実行になる。 <code>onPress={() => fn()}</code> と書く
<code>getMonth()</code>	0始まりなので +1 が必要
<code>padStart</code>	文字列メソッドなので <code>String()</code> で変換してから使う
<code>useEffect</code>	<code>onPress</code> の中には書けない
コンポーネントに <code>async</code>	つけられない。内部の関数に <code>async</code> をつける
ファイル名とルートパス	<code>kakeiboScreen.tsx</code> (コンポーネント) と <code>/(tabs)/kakeibo</code> (ルート) は別物
<code>router.replace</code> vs <code>push</code>	ログイン後は <code>replace</code> 、通常の遷移は <code>push</code>